

RELATÓRIO 02 — SUITE DE TESTES

M.I.N.D. — Muñdji Intelligent Neural Developer
Ponto 1 do plano de trabalho

Campo	Valor
Data	3 de Agosto de 2026
Ramo	claude/mind-prompt-implementation-9762wh
Commit	4f51da6
Pull request	Jos021/021 #1 (aberto e actualizado nesta etapa)
Dimensão	14 ficheiros, 1 537 linhas
Resultado	146 passam · 4 saltados · 1 xfail (151 recolhidos), em ~20 s

1. O que fiz

Executei o ponto 1 do plano apresentado no relatório anterior: converter a verificação avulsa feita durante o desenvolvimento — scripts soltos num directório temporário, que desapareciam com a sessão — numa suite versionada que corre com um comando e protege o comportamento já validado contra regressões futuras.

Abri também o pull request #1, conforme autorizaste, e actualizei-lhe o corpo depois desta etapa: a versão inicial dizia que não havia testes no repositório, o que deixou de ser verdade e passaria a induzir em erro quem revisse o PR.

1.1 Cobertura por área

Ficheiro	N.º	O que garante
test_sanitizador.py	27	Placeholders substituídos por valores realistas por tipo, IPs públicos trocados mas gamas privadas preservadas, acessos fora do workspace comentados, comentários sensíveis removidos sem partir código, e idempotência do filtro
test_hippocampus.py	24	Os princípios inegociáveis: cold-start seguro (sem dados, sem modelo, sem dependências ou com features inválidas devolve None e nunca lança), assimetria (auto-rejeição sim, auto-aprovação em nenhum caminho) e os fallbacks determinísticos das features
test_database.py	20	Modo WAL activo, foreign keys a impedir iterações órfãs, as sete tabelas, e a exportação JSONL filtrada por ciclo e por componente
test_ciclo.py	18	Fluxo ponta-a-ponta até à aprovação, marcadores mantidos durante o loop e removidos só na compilação final, validação cruzada, regra de divergência, penalização de marcadores órfãos, rollback e limite de iterações
test_sandbox.py	16	Execução real, timeout, isolamento das variáveis de ambiente do processo pai, confinamento ao workspace e degradação sem toolchain
test_marcadores.py	13	Parsing com anotação de linguagem, delimitação de secções, e a garantia de que a linguagem vem do marcador e nunca é inferida do conteúdo do código
test_ml_pipeline.py	13	Promoção por comparação, incluindo a garantia central de que um modelo pior nunca substitui o activo, e que fica registado no histórico
test_contratos.py	11	As verificações do contrato de interface, a reprovação apenas do NEURON violador e a justificação que o nomeia

Ficheiro	N.º	O que garante
test_activacao_dinamica.py	9	Todos os NEURONS na 1.ª passagem, só os visados a partir da 2.ª, ENABLE_NEURON_N como existência e não activação, e o circuit breaker a cortar um NEURON lento sem bloquear os restantes

2. Como fiz

2.1 Testes herméticos por construção

Cada teste recebe uma SYNAPSE DB e um workspace próprios em directório temporário. Uma fixture automática limpa as variáveis de ambiente que o MIND lê, para que a `.env` do utilizador nunca influencie o resultado. Nenhum teste depende da ordem de execução nem deixa resíduos.

O *RouterFalso* substitui as chamadas a modelos por respostas determinísticas por componente e regista quem foi chamado — é isso que permite afirmar, por exemplo, que um NEURON sem melhoria atribuída *não* é invocado, em vez de apenas verificar o resultado final.

2.2 Três decisões de método

Decisão	Razão
A lacuna conhecida virou um teste <i>xfail</i> estrito, não um comentário	<code>test_alteracao_fora_do_ambito_nao_e_detectada</code> descreve o comportamento correcto da verificação 3 do contrato e falha de propósito. Sendo estrito, passa a acusar erro no dia em que a lacuna for fechada — obriga a fechar o ciclo em vez de deixar a documentação envelhecer.
Testes de ML saltados, princípios de segurança testados sempre	Cold-start e assimetria são precisamente o que tem de se aguentar sem modelo e sem bibliotecas. Saltá-los quando faltam dependências esvaziaria o que interessa proteger.
Toolchains verificados por prova, não por presença no PATH	O <code>rustc</code> deste ambiente existe mas não tem toolchain por omissão. Um <code>shutil.which</code> dava falso positivo e o teste falhava por razão ambiental. Passei a compilar um programa trivial e a saltar só quando a ferramenta se revela inutilizável.

2.3 Falhas encontradas ao escrever os testes

As cinco falhas do primeiro arranque estavam nos **testes**, não no código do MIND — o que era esperado, já que este é o primeiro exercício sistemático sobre comportamento previamente verificado à mão:

- **Colisão de nomes.** Importei `test_coverage` de `ml_features` e o `pytest` recolheu-a como se fosse um teste. Passei a importá-la com outro nome.
- **Ordem de construção das fixtures.** O HIPPOCAMPUS lê `ML_MIN_TRAINING_SAMPLES` na construção, e a fixture é criada antes do `monkeypatch` do teste — quatro testes consultavam um componente que se julgava ainda em cold start.
- **Falso positivo de toolchain,** descrito acima.

A segunda merece nota para além do teste: em produção o HIPPOCAMPUS é construído depois de a `.env` ser lida, por isso não há defeito — mas o valor fica fixado na construção, o que significa que alterar o limiar exige reiniciar o processo. É uma característica, não um erro; fica registada aqui por ser o tipo de coisa que só se descobre a testar.

3. O que não fiz — e porquê

Item	Razão
Não corri com modelos LLM reais	Continua a ser a ressalva mais importante do projecto, e não mudou nesta etapa. Não há endpoints Ollama neste ambiente. A suite prova o esqueleto e as invariantes; não prova a qualidade dos prompts nem a robustez do parsing perante respostas de modelos reais.
Não medi cobertura de linhas	Escrevi os testes a partir dos requisitos da especificação, não a perseguir uma percentagem. Uma métrica de cobertura seria fácil de acrescentar, mas dá uma falsa sensação de segurança se as invariantes certas não estiverem testadas — e são essas que priorizei.
Não fechei nenhuma das lacunas do relatório anterior	Deliberado. A suite vinha primeiro precisamente para que as correcções seguintes — limpeza do git, verificação 3 do contrato, unificação do LangGraph — tenham uma rede a apanhá-las. Fechá-las antes seria trabalhar sem rede.
Não configurei integração contínua	O repositório não tem workflows e não foi pedido. A suite corre com <code>python -m pytest</code> ; ligá-la ao GitHub Actions é trivial se quiseres.

4. O que poderia fazer

- **Integração contínua** a correr a suite em cada push, para que uma regressão apareça no PR e não só na próxima vez que alguém se lembre de correr os testes.
- **Testes de propriedade** sobre o sanitizador e o parsing de marcadores — gerar entradas aleatórias e verificar invariantes (por exemplo: sanitizar nunca aumenta o número de IPs públicos).
- **Relatório de cobertura** para encontrar caminhos nunca exercitados, tratado como pista e não como objectivo.
- **Teste de integração com um modelo real**, saltado por omissão e activado por variável de ambiente quando houver um endpoint Ollama disponível.

5. O que vou fazer a seguir

Mantenho a ordem do plano do relatório anterior. Com a rede de segurança montada, os pontos 2 e 3 são agora seguros de atacar:

#	Acção	Estado
1	Suite de testes no repositório	Concluído nesta etapa (commit 4f51da6)
2	Corrigir <code>cleanup_temporary()</code> : implementar a limpeza real do histórico temporário mantendo os commits permanentes ($\geq 70\%$), ou alinhar a docstring com o que o código faz	Próximo. É o desvio mais claro face à especificação
3	Implementar a verificação 3 do contrato a sério — diff do código fora da secção atribuída. O teste <code>xfail</code> passa a verde	A seguir ao ponto 2
4	Unificar a execução no grafo LangGraph compilado, eliminando a reimplementação paralela em <code>MindGraph.run()</code>	Depende de 1-3 estarem estáveis
5	Tornar o parsing de relatórios robusto (saída estruturada em vez de regex sobre texto livre)	Primeiro ponto a partir quando entrarem modelos reais
6	Piloto com modelos reais via Ollama e preenchimento de <code>docs/model_selection_criteria.md</code>	Bloqueado por falta de endpoints

Vou avançar para o ponto 2 salvo indicação em contrário. Se tiveres endpoints Ollama a que eu possa chegar, diz — isso promove o ponto 6 ao topo, porque é o único que valida tudo o resto e porque as correcções seguintes beneficiam de se saber como os modelos reais se comportam.

Relatório gerado no fim da etapa. O resultado da suite indicado no cabeçalho foi obtido na execução final, após as correcções descritas em 2.3.